山东大学

01-可行性分析(研究)报告（FAR）

项目名称 博客系统

版　　本 V1.3

学　　院 计算机科学与技术学院

专　　业 计算机科学与技术

年　　级 2023 级

协作平台 GitHub

2026 年 03 月 13 日

目　　录

第 1 章　引言 4
1.1 标识 4
1.2 背景与问题陈述 4
1.3 项目概述 5
第 2 章　可行性分析的前提 6
2.1 需求概览 6
2.1.1 功能需求 6
2.1.2 非功能需求 6
2.2 假定、约束和边界 6
2.3 分析方法 7
2.4 主流 CASE 工具调研与选型 8
2.4.1 调研范围与分类 8
2.4.2 主流 CASE 工具调研分析 8
2.4.3 本项目 CASE 工具选型与应用 9
2.4.4 工具协同流程 10
第 3 章　可选方案与比较 11
3.1 方案列表 11
3.2 方案对比 11
3.3 开发模型选型分析 12
第 4 章　建议系统与逻辑/流程模型 13
4.1 系统边界与参与者（上下文关系图） 13
4.2 系数据流与核心闭环（DFD 0 层） 14
4.3 分区与子分区关系模型（结构关系图） 15
4.4 建区申请与审核（状态机 + 流程） 15
4.4.1 建区申请单状态机（一级/二级同用） 15
4.4.2 建区申请处理流程（泳道/流程图） 16
4.5 分区内模块（板块）与内容类型（关系图） 17
4.6 用例模型（用户/管理员能做什么） 17
4.7 核心实体 E-R 逻辑图 18
4.8 可扩展想法（写进 FAR 的“建议系统”会更专业） 18

第 5 章　可行性评估 20
5.1 技术可行性 20
5.1.1 技术可行性 20
5.1.2 关键技术风险与对策 20
5.2 法律与合规可行性 20
5.3 用户与组织可行性（使用、运营、维护） 21
5.3.1 用户使用可行性 21
5.3.2 运营与管理可行性 21
5.3.3 维护可行性 21
5.4 风险管理与应对预案 21
5.4.1 分级风险标准 21
5.4.2 细化风险清单与分级应对预案 22
5.4.3 风险监控与迭代优化 24
5.4.4 数据增长与性能 24
5.4.5 验收可度量指标 24
5.5 注解（术语表） 24
第 6 章　附录 A：MVP 与迭代建议 25
第 7 章　附录 B：引用文件 27
第 8 章　附录 C：Scrum 开发过程模型落地设计 28
8.1 Scrum 核心角色分配 28
8.2 Scrum 核心活动执行规范（适配课程项目） 28
8.3 Scrum 核心工件管理 28

第1章　引言

1.1 标识

系统名称：BlogZone（分区可配置的网络博客系统）

系统类型：B/S 架构 Web 应用（可扩展移动端）

文档类型：可行性分析(研究)报告（FAR）

1.2 背景与问题陈述

传统的博客平台多依赖“分类”或“标签”对内容进行组织，尽管这种模式在一定程度上满足了内容归类的需求，但在面对多主题、多场景融合的社区化发展趋势时，逐渐暴露出结构上的局限性。具体而言，不同主题对内容呈现形式与功能模块有着差异化的需求，例如摄影类用户更倾向于图文并茂的作品展示形态，求职类板块则需要具备招聘信息发布与简历投递功能，校园场景往往涵盖活动组织和二手交易等多种互动方式，而技术社区则更关注问答系统和资料库的建设。单一的文章列表模式难以承载如此多元的功能诉求，无法实现“分区差异化”的内容与功能配置。此外，随着社区生态的发展，新的主题不断涌现，要求平台具备“可生长”的架构能力，能够动态扩展新的内容分区，并建立配套的审核机制与治理体系以确保分区质量和社区秩序。

基于上述背景，本文提出构建一个具备高度可扩展性的通用博客平台，其核心设计思路如下：以“分区（Zone） / 子分区（SubZone”作为内容与功能的组织单元，支持在分区内灵活配置功能板块（Module），实现不同主题下的差异化功能呈现；同时构建完整的互动机制与社区治理闭环，保障用户体验与内容质量；并设计分区申请创建与审核流程，赋予社区自我演进和有序生长的能力，最终形成一个结构灵活、功能可配、治理规范的新型博客社区架构。

1.3 项目概述

本系统的设计目标是在课程项目的时间与资源约束下，构建一个功能完备、结构清晰且具备可扩展能力的博客社区原型系统，确保各项核心功能能够落地实现并形成完整的业务闭环。在用户层面，系统需支持用户的注册与登录功能，并提供个性化的个人主页，涵盖头像、背景图、主题风格及用户 ID 等基础配置；用户应能够发布图文内容，浏览不同分区下的文章列表，并对内容进行点赞、评论、转发及收藏等互动操作；此外，系统需引入 VIP 会员体系，支持 VIP 标识的展示及模拟开通流程，以体现用户等级的差异化体验。在平台治理方面，系统需内置必要的管理功能，包括内容置顶、用户举报处理、内容下架操作以及相应的审核日志记录，确保平台内容符合规范并具备可追溯的治理能力。在分区体系设计上，系统支持一级分区与二级分区的层级结构，每个分区内部可根据主题需求灵活配置功能模块，初期至少实现“帖子”与“公告”两类模块，并预留扩展接口以支持后续如问答、二手交易、活动组织和资料库等模块的接入。为体现社区的可生长特性，系统需设计分区增长机制，允许用户提交创建一级或二级分区的申请，申请经审核员审核通过后，系统自动完成分区的创建及初始模块的配置，并将申请者授权为该分区的主理人或管理员，赋予其一定的管理权限，从而构建起从用户申请到平台审核再到分区上线的完整闭环流程。通过上述设计，本系统旨在实现一个功能可配、结构可扩展、治理可追踪的通用博客社区平台，既满足课程项目的落地要求，又为后续功能的持续迭代奠定基础。

第2章　可行性分析的前提

2.1 需求概览

2.1.1 功能需求

- 用户：注册/登录、个人主页（头像/背景/主题、昵称与唯一 ID）、关注/收藏（可选）、VIP（等级/有效期/权益展示）
- 内容：发布图文帖子；编辑/删除；按分区/子分区归属；置顶（按权限）
- 互动：点赞、评论、转发（可选）、通知中心（可选）、举报闭环
- 分区：多分区、支持二级；分区内模块（板块）可启用/禁用/排序
- 分区创建：用户申请创建一级/二级分区；审核员审核通过后自动创建并初始化配置

2.1.2 非功能需求

- 安全：鉴权、权限控制、密码加密、上传校验、基础防 XSS/越权
- 可用性：流程清晰（浏览→发布→互动→治理），新用户易上手
- 可维护性：分层架构、配置化模块、日志与审计可追溯
- 可扩展性：新增分区、增加模块类型对已有系统影响小

2.2 假定、约束和边界

基于课程项目的定位与可行性考量，本系统的设计与实现将遵循以下原则，以确保项目在有限周期内能够高质量落地并充分体现工程规范：

首先，在系统规模方面，项目以课程演示为目标，不追求高并发场景下的极端性能表现，而是将重点放在系统的可运行性、功能完整性以及工程化标准的贯彻上，确保演示过程流畅、逻辑闭环清晰。涉及支付功能的场景，系统将采用“模拟订单状态”的方式进行实现，即通过前端状态切换或后端简易逻辑模拟订单的创建、支付成功、支付失败等流程，而不对接真实支付渠道，既满足业务流程的演示需求，又规避了复杂性与安全风险。

在分区体系设计上，为控制树形结构的复杂度并降低实现难度，系统明确将分区层级限定为两级，即一级分区与二级分区。该设计既能满足大多数场景下的内容归类需求，又可避免因无限层级嵌套带来的递归查询、权限继承、界面展示等方面的工程复杂性，确保数据结构简洁、查询高效、逻辑清晰。

针对分区内功能模块的配置需求，系统采用“配置驱动”的实现思路，而非构建完整的插件化体系。具体而言，通过在后台预先定义若干模块类型（如帖子列表、公告栏等），并在分区创建或编辑时以开关或选择框的形式进行模块启用与排序配置，从而实现不同分区呈现不同功能形态的目标。该方案在满足分区差异化需求的同时，避免了插件系统所需的动态加载、接口隔离、生命周期管理等复杂设计，大幅降低工期风险与实现难度，确保项目核心功能优先落地、稳定运行。

综上，本系统将在规模可控、支付模拟、层级限定、配置驱动等原则指导下，以工程规范为准绳，构建一个结构清晰、功能完整、易于演示与评估的博客社区原型系统。

2.3 分析方法

为确保系统设计的严谨性与可落地性，本项目将采用“方案对比 + 风险清单 + 闭环验证”三位一体的系统化分析方法。在方案设计阶段，针对关键功能点（如分区模块配置方式、审核流程状态机设计等）将提出多种可选实现方案，并从技术复杂度、开发成本、可扩展性等维度进行综合对比，为决策提供客观依据；同时，针对每一设计方案提前识别潜在的技术实现风险与业务逻辑漏洞，形成风险清单并制定相应的应对策略或备选方案，以降低项目实施过程中的不确定性。在需求验证阶段，将围绕核心业务流程构建完整的闭环验证路径，分别从用户视角与管理视角梳理关键操作链路，确保系统功能覆盖从发起、处理到反馈的完整环节。为增强需求的可沟通性、可实现性与可验收性，本阶段还将输出多种类型的关系图与流程图，包括但不限于实体关系图（ER 图）、用户操作流程图、

管理后台审核流程图、状态流转图等，以图形化方式直观呈现系统结构、业务逻辑与交互流程，便于项目团队成员之间的理解对齐，并为后续的系统设计、开发实现与测试验收提供统一、明确的参考依据。通过上述方法的系统化运用，本项目旨在构建一个逻辑严密、风险可控、易于沟通与落地的博客社区平台设计方案。

在 “方案对比 + 风险清单 + 闭环验证” 三位一体分析方法基础上，同时融入敏捷 Scrum 核心思维，以 “快速交付可运行产品、持续响应需求调整” 为导向，将核心功能拆解为可迭代的冲刺任务，通过闭环验证确保每个迭代产出物满足课程验收要求；同时结合风险清单提前识别迭代过程中的技术、进度风险，通过 Scrum 的每日站会、冲刺评审及时发现并解决问题，确保项目在课程周期内高质量落地，兼顾工程规范性与开发灵活性。

2.4 主流 CASE 工具调研与选型

2.4.1 调研范围与分类

小组对教材及主流软件工程技术网站（CSDN、思否、开源中国、微软开发者中心、Oracle 技术社区）进行调研，覆盖 UML 建模、数据库设计、流程绘图、项目管理、配置管理、测试工具六大类 CASE 工具，重点分析用途、技术特点与课程项目适配性。

2.4.2 主流 CASE 工具调研分析

工具类别	工具名称	核心用途	技术特点	适用场景
UML / 流程建模	Microsoft Visio	绘制 DFD、ER 图、流程图、泳道图、甘特图	模板丰富、拖拽易用、与 Office 深度集成	需求分析、系统设计、流程可视化
	StarUML	标准 UML 建模（用例图、类图、状态机、序列图）、ERD/DFD	支持 UML2.x、轻量高效、可导出代码骨架	面向对象设计、系统逻辑建模
	PlantUML	文本脚本生	开源、文本化、可	敏捷文档、快速

工具类别	工具名称	核心用途	技术特点	适用场景
		成 UML 图、文档内嵌图表	Git 版本管理、适配 Markdown	绘图、团队协作
	Enterprise Architect	全生命周期建模、需求 - 设计 - 代码追踪	企业级、支持双向工程、团队仓库	大型项目、复杂系统建模
数据库设计	Sybase PowerDesigner	CDM/PDM 建模、SQL 生成、反向工程、数据字典	支持主流数据库、模型联动、团队协作	MySQL/Oracle 等库表设计
配置管理	Git	分布式版本控制、分支管理、代码合并	开源、高效、支持 GitHub/GitLab 协作	全周期代码管理、敏捷开发
	Visual SourceSafe	早期集中式版本控制	简单易用、文件锁定	老旧小型项目、已逐步淘汰
	ClearCase	企业级配置管理、基线管控	权限严密、适合大型团队	超大型项目、强合规场景
项目/进度管理	TFS	项目管理 + 版本控制 + 构建集成	微软生态、适配瀑布 / 敏捷	Windows 技术栈团队
测试工具	Apache JMeter	接口压力测试、性能基准测试	开源、支持 HTTP/DB、生成报告	后端接口、并发性能验证

2.4.3 本项目 CASE 工具选型与应用

结合博客系统（前后端分离 + Scrum 敏捷） 技术路线与课程项目特点，确定轻量化、开源 / 免费、易协作工具组合，覆盖全开发流程：

（1）建模与设计

StarUML：绘制用例图、ER 图、状态机、DFD 图，支撑系统逻辑设计。

PlantUML：快速生成流程图、组件图，嵌入文档并 Git 托管。

Visio：绘制泳道图、甘特图、系统上下文图，用于报告与汇报展示。

（2）数据库设计

PowerDesigner：完成 MySQL 概念 / 物理模型设计，自动生成建表 SQL 与数据字典。

（3）配置管理

Git + GitHub：代码版本控制、分支管理、Pull Request 代码审查，支撑 Scrum 迭代协作。

（4）项目与进度管理

甘特图（Visio/在线工具）：规划 Sprint 周期、任务排期、里程碑管控。

（5）测试验证

Apache JMeter：对发帖、评论、分区查询等核心接口做性能测试，保障稳定运行。

2.4.4 工具协同流程

需求/设计阶段：用 StarUML/PlantUML/Visio 完成建模 → 输出设计图与报告。

开发阶段：Git 管理代码 → 按 Sprint 分支并行开发 → 合并验证。

数据库阶段：PowerDesigner 设计表结构 → 生成 SQL 脚本 → 导入 MySQL。

测试阶段：JMeter 做接口 / 性能测试 → 输出测试报告，支撑验收。

文档阶段：图表统一版本化，与代码同步迭代，确保一致性。

第3章 可选方案与比较

3.1 方案列表

方案 A：直接使用现成博客/论坛平台（如 WordPress、Discuz 等）

方案 B：单体应用 + 前后端分离 + 配置驱动模块（建议）

方案 C：微服务或插件化平台（工程更复杂）

3.2 方案对比

维度	方案 A：现成平台	方案 B：单体前后端分离（推荐）	方案 C：微服务/插件化
开发成本	最低	中等（可控）	高
文档对齐课程要求	较差（偏配置部署）	好（需求-设计-实现闭环）	好但工作量大
分区+模块可配置实现	需深度二开	原生设计即可	可做得最好
风险	二开复杂度不确定	可控、适合迭代	联调部署风险高
交付把握	中	高	低-中

建议结论：基于上述多维度对比分析，本项目建议选用方案 B（单体应用 + 前后端分离 + 配置驱动模块）作为核心实施路线。该方案在开发成本与工程规范性之间取得了最佳平衡：相较于方案 A，方案 B 能够完全自主掌控系统架构，从需求分析、系统设计到代码实现形成完整闭环，更好地对齐课程文档要求，避免现成平台“重配置、轻开发”带来的设计与实现脱节问题；相较于方案 C，方案 B 规避了微服务架构带来的服务拆分、分布式事务、跨服务联调等复杂性问题，将工程风险控制在可承受范围内，确保项目在有限周期内能够高质量交付。在核心功能实现上，方案 B 以“配置驱动”为设计理念，通过在单体架构内预先定义模块类型并支持分区级别的模块开关与排序配置，即可满足分区差异化功能需求，同时为后续扩展预留清晰接口。项目实施策略上，建议进一步明确 MVP（最小可行产品）边界，优先实现帖子、公告等核心模块，以及用户注册登录、分区浏览、发文互动、审核治理等基础闭环功能，确保演示版本可运行、可演示、可评估；在此基础上，以可迭

代的方式逐步扩展问答、二手交易、活动组织等进阶模块，既控制初期开发风险，又体现系统的可生长特性。综上，方案 B 兼具开发可控性、需求匹配度与扩展灵活性，是本项目最为务实可行的技术选型。

3.3 开发模型选型分析

维度	传统模型（如瀑布模型）	敏捷开发（如 Scrum, XP）
特点	线性、阶段明确、文档驱动	迭代、增量、响应变化、用户参与
优点	进度易控，适合需求极度稳定的项目	交付快，容错率高，能快速验证核心功能（MVP）
缺点	难以应对需求变更，后期发现问题成本极高	弱化文档可能导致后期维护困难，对成员自律要求高

结合博客系统项目课程实验属性、MVP + 迭代的功能规划、小团队协作的特点，确定采用 Scrum 敏捷开发方法作为核心开发模型。

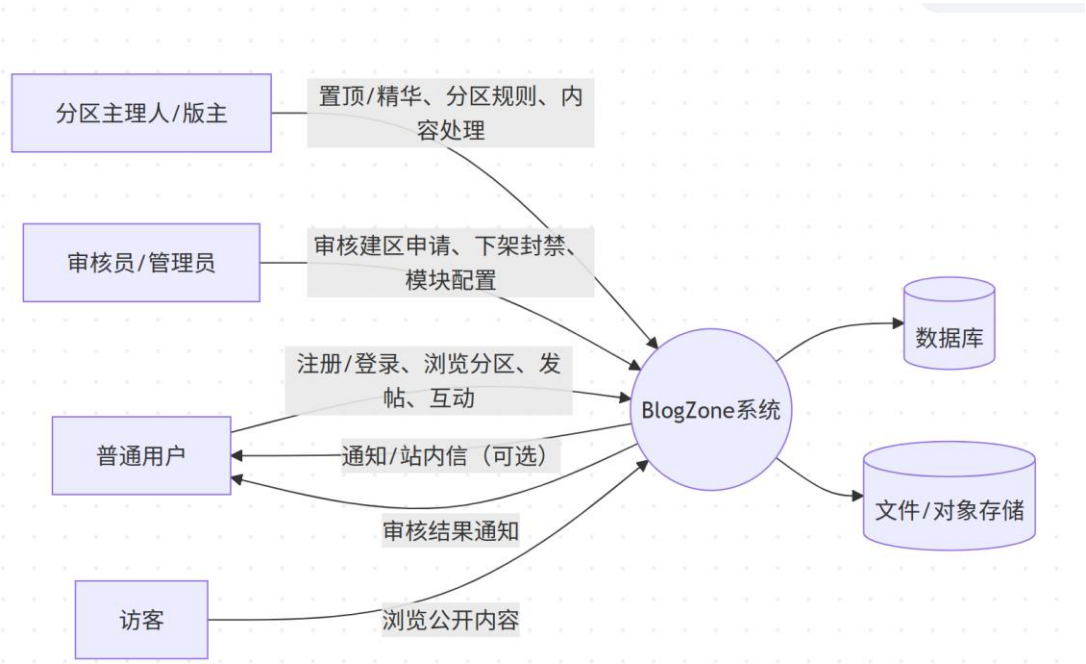
Scrum 在本项目中的具体应用规划有：

1. 迭代周期规划：将整体开发划分为 3 个 Sprint 冲刺，每个冲刺周期 2 周，均以 “交付可运行、可演示的功能版本” 为目标；
2. 产品待办列表：将附录 A 中的 “必做核心功能”“迭代增强功能” 拆解为粒度可执行的任务，按优先级排序（核心闭环功能优先级最高，扩展功能次之）；
3. 冲刺待办列表：每个冲刺开始前，从 Product Backlog 中选取可完成的任务，明确冲刺目标与交付标准；
4. 适配课程需求：冲刺结束后同步完成文档迭代，弥补敏捷轻文档的不足，满足课程对可行性报告、设计文档的规范性要求。

第4章 建议系统与逻辑/流程模型

本节为 FAR 的“专业度核心”：用多张逻辑/流程/关系图，证明系统能落地、边界清晰、可治理。

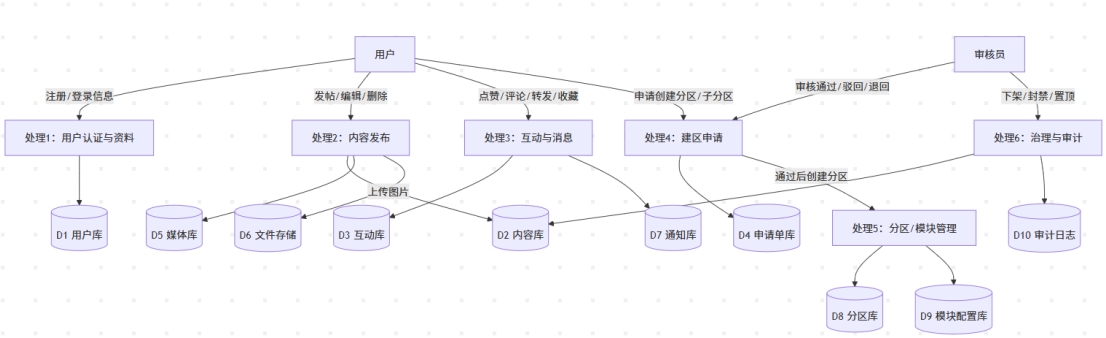
4.1 系统边界与参与者（上下文关系图）



本系统的用户角色体系围绕内容生产、社区治理与平台运营三大核心职能进行设计，涵盖四类主要用户群体，分别为普通用户、分区主理人/版主、审核员/管理员以及访客。普通用户是平台内容创作与互动的主力军，通过注册登录后可浏览各分区内容、发布图文帖子并对内容进行点赞、评论、转发及收藏等操作，深度参与社区生态。分区主理人（或称版主）作为各分区的实际管理者，拥有该分区内的内容管理权限，包括置顶优质内容、标记精华帖、制定分区规则以及对违规内容进行处理，承担着维护分区秩序与活跃度的职责。审核员/管理员则站在平台全局视角，负责审核用户提交的一级或二级分区创建申请，对违规内容进行下架或封禁处理，并对各分区的功能模块进行配置管理，保障平台整体内容合规与系统稳定运行。此外，访客作为平台的轻度参与者，无需注册即可浏览公开分区的内容，体验平台的阅读功能，成为潜在的用户转化来源。上述四类用户通过前端界面与系统核心

（BlogZone 系统）进行交互，系统在处理各类业务请求时，一方面通过通知或站内信机制（作为可选功能）向用户反馈审核结果或系统消息，另一方面将用户数据、内容数据、分区配置信息等持久化存储于关系型数据库中，同时将用户上传的图片、附件等非结构化数据存放于文件存储或对象存储服务中，形成完整的数据存储与访问体系。通过上述角色划分与系统交互设计，本系统构建起一个权限清晰、职责明确、数据闭环的通用博客社区平台架构。

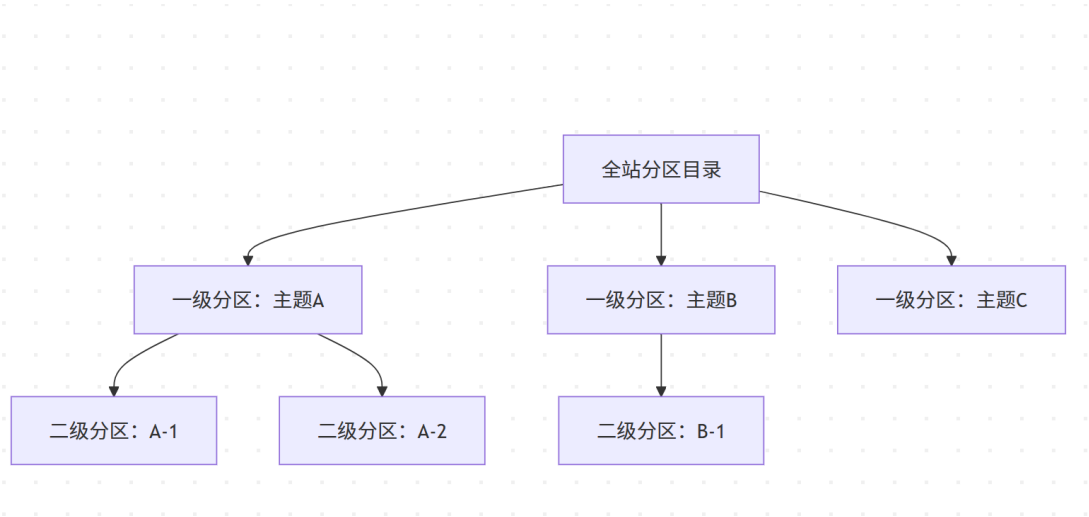
4.2 系数据流与核心闭环（DFD 0 层）



本系统的核心业务处理流程围绕用户操作与后台管理两条主线展开，涵盖内容生产、社区互动、分区建设与平台治理四大关键环节，形成完整的数据流转与处理闭环。在用户侧，系统以用户认证为起点，通过“处理 1：用户认证与资料”模块完成注册、登录及个人信息管理功能，并将用户基础数据持久化至用户库（D1）中。用户在完成认证后，可进入内容创作流程，通过“处理 2：内容发布”模块进行发帖、编辑及删除操作，系统在此过程中将图文内容存储至内容库（D2），同时将用户上传的图片等媒体文件分别存入媒体库（D5）与文件存储（D6），实现结构化数据与非结构化数据的分类管理。用户在浏览内容时产生的点赞、评论、转发及收藏等互动行为，由“处理 3：互动与消息”模块统一接收和处理，一方面将互动记录写入互动库（D3），另一方面根据业务规则生成系统消息或通知，存入通知库（D7）以备用户查收。作为平台可生长特性的核心体现，用户可通过“处理 4：建区申请”模块提交一级或二级分区的创建申请，相关数据暂存于申请单库（D4）；审核员登录后台对申请进行审核，作出通过、驳回或退回修改等决策，审核通过

后系统自动触发“处理 5：分区与模块管理”模块完成分区的实际创建及初始功能模块的配置，并将分区结构与模块配置信息分别写入分区库（D8）与模块配置库（D9），同时将申请者授权为分区主理人。在平台治理层面，审核员通过“处理 6：治理与审计”模块对违规内容进行下架、封禁或置顶等操作，相关处理记录一方面同步更新内容库（D2）中的内容状态，另一方面写入审计日志库（D10），形成可追溯的审核记录，满足平台合规运营与事后审计的需求。通过上述六大处理模块与十类数据存储的协同配合，本系统构建起从用户操作到后台响应、从数据写入到状态流转的完整业务闭环，为博客社区的高效运行与规范治理提供了坚实的技术支撑。

4.3 分区与子分区关系模型（结构关系图）

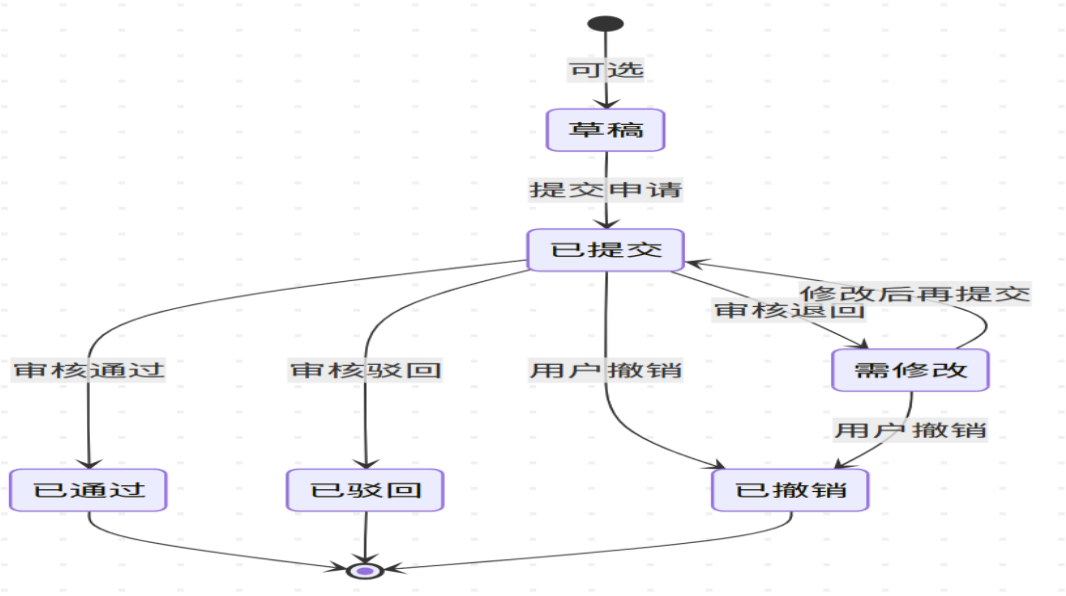


设计要点（可行性角度）：

- 两级结构实现简单（数据库 parent_zone_id），导航与权限也更可控
- 后续若要支持多级，可在不破坏现有结构的前提下扩展为树形查询

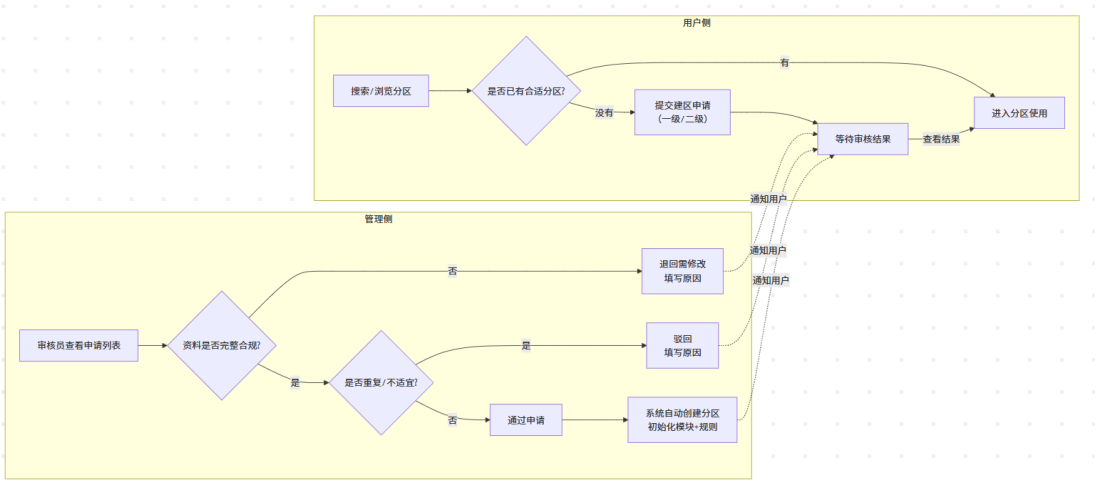
4.4 建区申请与审核（状态机 + 流程）

4.4.1 建区申请单状态机（一级/二级同用）



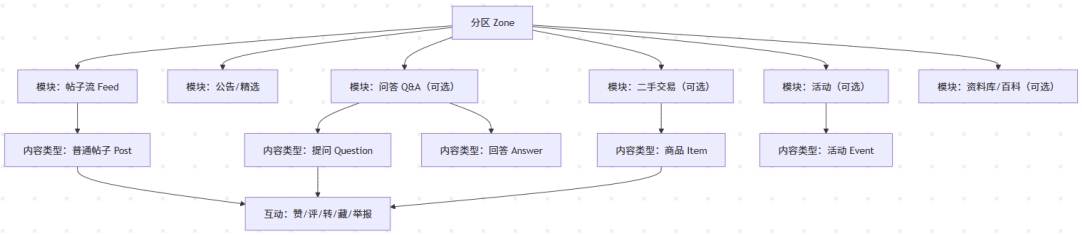
本状态图完整呈现了用户申请创建分区时的全生命周期流转逻辑。申请以“草稿”状态为起点（可选步骤），用户提交后进入“已提交”状态，正式进入审核流程。审核员在处理申请时可根据情况作出三种决策：若资料存在问题但可修改，则退回至“需修改”状态，允许用户调整后重新提交；若申请不符合平台规则，则直接“已驳回”终止流程；若资料完整合规，则“已通过”并触发分区自动创建。此外，用户在任何非终态阶段均可主动撤销申请，流程随之终止。通过上述状态机设计，系统实现了申请流程的规范化管理与全链路可追溯，确保分区创建过程清晰可控。

4.4.2 建区申请处理流程（泳道/流程图）



本流程图清晰展示了用户申请创建分区的完整业务流程，涵盖用户侧与管理侧两大核心环节。在用户侧，用户首先搜索或浏览现有分区，若未找到符合需求的分区，则提交一级或二级分区创建申请，随后进入等待审核状态。在管理侧，审核员查看申请列表后依次进行资料完整性检查和内容适宜性判断：若资料不完整则退回并注明修改原因；若申请重复或不符合平台定位则驳回并说明理由；若资料完整且内容适宜则予以通过。申请通过后，系统自动完成分区创建及初始功能模块与规则的初始化配置。用户侧与管理侧之间通过双向通知机制实现信息闭环，审核过程中的退回、驳回或通过结果均实时通知用户，确保流程透明、反馈及时。该流程图完整呈现了从用户发起申请到平台审核处理再到系统自动落地的业务闭环，体现了分区创建流程的规范化与自动化设计。

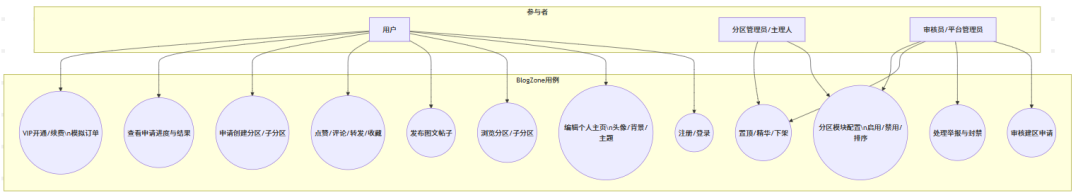
4.5 分区内模块（板块）与内容类型（关系图）



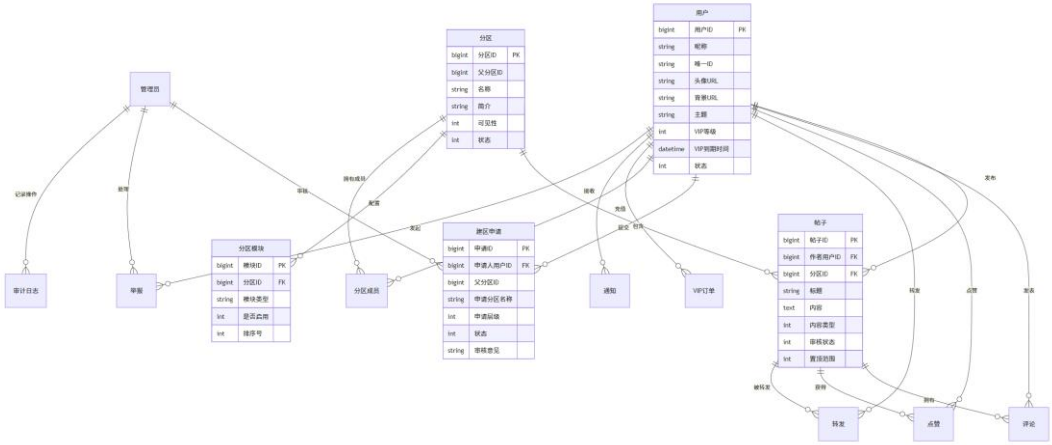
说明：

- “模块”解决分区页面呈现与能力差异
- “内容类型”解决数据结构差异（字段不同）
- 互动能力（赞评转藏举报）尽量抽象为通用事件模型，降低重复开发成本

4.6 用例模型（用户/管理员能做什么）



4.7 核心实体 E-R 逻辑图



本实体关系图（ER 图）完整呈现了博客社区系统的核心数据模型与业务实体间的逻辑关联。系统以用户实体为中心，其与帖子、评论、点赞、转发等实体形成内容生产与互动链路：用户可发布帖子，帖子归属于特定分区，其他用户可对帖子进行评论、点赞及转发操作，这些互动行为分别与相应用户建立关联，形成完整的内容生态闭环。在社区治理与扩展机制层面，用户可提交建区申请，由管理员进行审核处理，审核通过后系统自动创建分区并关联分区模块配置，实现分区功能的差异化呈现；同时，用户可通过分区成员实体与分区建立关联，成为分区主理人或普通成员，承接分区的日常管理权限。在平台运营与合规维度，VIP 订单记录用户的会员充值信息，通知实体承载系统消息与审核结果的推送，举报实体支持用户对违规内容发起投诉并由管理员处理，所有管理操作均通过审计日志进行记录，确保平台治理过程可追溯。上述实体通过明确的主外键约束与关系基数定义，构建起一个结构清晰、覆盖全面的数据模型，为系统的功能实现与业务扩展提供了坚实的底层支撑。

4.8 可扩展想法（写进 FAR 的“建议系统”会更专业）

以下为“可添加但不强依赖”的设计想法，用于证明系统具备可扩展性（同时不影响 MVP 交付）：

- 分区模板（Template）

- 新分区创建时选择模板：如“通用讨论模板 / 交易模板 / 问答模板 / 作品展示模板”
- 模板决定默认模块组合、发帖规则、审核策略（先发后审/先审后发）
- 分区生命周期管理
- 分区状态：正常 / 冻结（只读）/ 合并 / 已关闭
- 冻结触发：长期无人发帖、违规高发、无主理人维护等
- 合并能力：将小分区合并到相似大分区，避免分区膨胀导致体验下降
- 分区自治与多级治理（可选）
- 二级分区申请可由父分区管理员初审，平台兜底抽检
- 课程实现可先统一走平台审核，设计中预留扩展点
- 内容推荐与热度计算（可解释规则）
- 热度 = 点赞权重 + 评论权重 + 转发权重 + 时间衰减
- 支持“精华”人工干预（版主/管理员）
- 安全与风控增强（可选）
- 反滥用：建区申请频率限制、发帖频率限制
- 敏感词：标题/正文敏感词过滤（简化版即可）
- 图片审核：先不做 AI 审核，保留人工举报渠道
- 更完整的通知体系
- 通知类型：评论、回复、点赞、转发、被关注、申请审核结果、系统公告
- 支持已读/未读、批量已读、通知聚合

第5章 可行性评估

5.1 技术可行性

5.1.1 技术可行性

本系统属于典型“Web 社区信息系统”，常见技术栈成熟：

- 前端：Vue3/React + UI 组件库
- 后端：Spring Boot / Django / NestJS 等
- 数据库：MySQL/PostgreSQL
- 文件存储：本地或 MinIO/对象存储
- 接口：RESTful + Swagger/OpenAPI
- 权限：JWT/Session 方案均可（课程建议简化但要规范）

5.1.2 关键技术风险与对策

风险点	描述	对策（可行性角度）
权限越权	置顶/下架/模块配置等敏感操作	统一鉴权中间件 + 资源级权限校验（作者/分区管理员/平台管理员）
上传安全	木马伪装、超大文件、路径穿越	白名单类型、大小限制、重命名存储、独关键技术风险与对策立静态目录、必要时对象存储
互动幂等	重复点赞/重复请求导致计数错误	唯一约束（user_id, post_id）+ 幂等接口
审核闭环复杂	多状态、多角色操作	采用状态机设计（图 5-4）+ 审计日志
模块扩展失控	想做太多模块导致延期	明确 MVP：帖子+公告+建区审核为必做，其余迭代

结论：技术风险可控，技术可行

5.2 法律与合规可行性

- 版权风险：UGC 可能侵权 → 提供举报/下架通道与处理日志
- 隐私保护：密码哈希存储；最小化采集个人信息；后台访问留痕
- 内容合规：支持封禁、禁言、下架、审计日志；课程阶段可先实现基础闭环
- 支付合规：采用模拟订单不接真实支付，风险低

结论：在课程项目范围内法律合规可行。

5.3 用户与组织可行性（使用、运营、维护）

5.3.1 用户使用可行性

典型用户路径清晰：

- 访客：浏览公开分区 → 查看帖子详情
- 用户：注册登录 → 选择分区 → 发布图文 → 互动 → 个人主页展示
- 找不到分区：提交建区申请 → 接收结果通知 → 成为分区主理人

5.3.2 运营与管理可行性

- 管理侧有明确工作台：建区申请列表、内容举报列表、分区配置、下架封禁
- 审计日志可追溯：谁在何时对何对象做了何操作，便于验收展示“治理能力”

5.3.3 维护可行性

- 分层架构与配置驱动降低维护成本
- 数据模型清晰（图 5-8），便于后续扩展模块类型

结论：用户与组织层面可行。

5.4 风险管理与应对预案

5.4.1 分级风险标准

本项目基于发生概率（高 / 中 / 低）和影响程度（高 / 中 / 低）构建二维风险分级矩阵，将风险划分为高风险（红色）、中风险（黄色）、低风险（蓝色） 三个等级，其中高风险为重点管控对象，需制定专项应对预案；中风险为常规管控，需明确应对措施与责任人；低风险为观察管控，需建立预警机制。

5.4.2 细化风险清单与分级应对预案

风险编号	风险项	发生概率	影响程度	风险等级	具体描述	应对预案	责任主体
R01	核心功能开发延期	高	高	红色	过度追求扩展功能（如问答 / 二手模块），导致建区审核、内容治理等核心闭环功能无法按期交付	1. 严格执行 MVP 原则，冲刺阶段仅开发当前优先级最高的核心功能； 2. 若进度落后，立即砍掉非核心功能（如 VIP 模拟、转发功能），仅保留“建区申请 - 审核 - 发帖 - 治理”核心闭环； 3. 每日站会同步进度，发现任务阻塞立即协调团队资源攻坚	全团队
R02	分区权限控制实现困难	中	高	黄色	一级 / 二级分区的权限继承、分区主理人操作权限校验逻辑复杂，出现越权漏洞或实现卡顿	1. 技术降级设计：先实现**资源级权限校验**（按 user_id/zone_id 做基础判断），暂不实现复杂的权限继承； 2. 提前编写权限测试用例，开发过程中边做边测，避免后期集中排查； 3. 若技术攻关超时，采用“固定分区模板 + 简易权限”替代动态配置，保证功能可用	后端开发组
R03	互动功能出现幂等性问题	中	中	黄色	点赞、评论等互动操作出现重复计数，或删除 / 编辑操作触发异常	1. 技术层面：添加**唯一约束（user_id+post_id/comment_id）**，设计幂等接口，避免重复请求； 2. 测试层面：冲刺评审阶段重点测试互动功能的边界场景（多次点击、快速操作）； 3. 预留修复时间：每个冲刺最后 1 天为 bug 修复期，专门解决此类技术问题	前后端开发组
R04	团队协作代码冲突	中	中	黄色	多人同时开发同一模块（如用户中心、分区管理），GitHub 提交代码时出现严重冲突，导致开发停滞	1. 规范 GitHub Flow 协作流程：每人基于 dev 分支创建个人功能分支，仅在功能完成后提交 Pull Request； 2. 代码审查机制：由技术负责人审核 Pull Request，解决冲突后再合并至 dev 分支； 3. 每日同步代码：团队成员每日下	全团队

风险编号	风险项	发生概率	影响程度	风险等级	具体描述	应对预案	责任主体
						班前拉取 dev 分支最新代码，及时解决小型冲突	
R05	UGC 内容合规风险	低	中	黄色	用户发布的图文内容出现敏感信息，平台缺乏有效过滤机制	1. 简化版敏感词过滤：开发基础的敏感词匹配功能，对标题 / 正文进行初步校验； 2. 强化人工治理：完善“举报 - 下架 - 审计”闭环，确保违规内容可快速处理； 3. 课程演示阶段：采用模拟测试数据，避免真实敏感内容出现	前端开发组 + 管理模块开发组
R06	界面与交互体验不佳	低	低	蓝色	分区浏览、发帖等核心操作的界面流程繁琐，新用户上手困难	1. 前期完成低保真原型设计，确认核心操作流程≤3 步； 2. 冲刺评审阶段邀请同学模拟测试，收集体验反馈并快速优化； 3. 优先保证功能可用，界面优化仅做基础调整，不追求极致效果	前端开发组
R07	文档与开发不同步	低	低	蓝色	敏捷开发过程中忽略文档更新，导致可行性报告、设计文档与实际开发不一致	1. 文档迭代与开发同步：每个冲刺结束后，1 天内完成对应模块的文档更新； 2. 指定文档负责人，专门负责文档的整理、校对与更新； 3. 课程验收前 3 天，全团队统一核对文档与代码的一致性	文档负责人 + 各模块开发人

5.4.3 风险监控与迭代优化

1.建立每日风险预警机制：每日站会中增加 “风险同步” 环节，各成员反馈当前遇到的风险或潜在风险，由 Scrum Master 记录并跟踪；

2.冲刺评审风险复盘：每个冲刺结束后，对本阶段出现的风险进行复盘，分析原因并优化应对预案，避免后续重复出现；

3.动态更新风险清单：项目开发过程中，若出现新的风险点，及时补充至风险清单并评估等级，制定对应的应对措施。

5.4.4 数据增长与性能

- 通过分页、索引（post.zone_id、post.created_at 等）、缓存（可选）保障基本性能
- 演示规模足够；若扩展生产需进一步做读写分离、缓存、异步化等

5.4.5 验收可度量指标

- 功能闭环：用户可完成“建区申请→审核→分区生成→发帖→互动→治理”
- 安全性：未登录不可发帖；普通用户不可置顶/下架；密码不可明文
- 可用性：核心操作≤3 步可达（进入分区、发帖、互动）

5.5 注解（术语表）

- 分区（Zone）：主题容器，可一级/二级
- 模块（Module）：分区内功能板块，如帖子/公告/问答/二手
- 建区申请：用户申请创建分区/子分区的流程单
- 治理：置顶、精华、下架、封禁、举报处理、审计日志
- MVP：最小可行产品，保证按期交付的最小功能集合

第6章附录 A：MVP 与迭代建议

Sprint 1（第 1-2 周）：基础功能落地，实现内容核心闭环

1. 用户注册登录、个人主页基础信息配置（头像 / 背景 / 昵称 / 唯一 ID）
2. 一级 / 二级分区基础浏览、分区列表与详情展示
3. 图文发帖、编辑 / 删除、内容列表与详情页呈现
4. 基础互动功能：点赞、一级评论（核心互动闭环）
5. 数据库核心表设计与实现，基础接口开发

Sprint 2（第 3-4 周）：治理功能落地，实现平台生长闭环

1. 建区申请与审核流程（提交 - 审核 - 通过 / 驳回 - 自动创建分区 + 初始化模块）
2. 后台治理核心功能：内容下架 / 置顶、审计日志基础记录（操作人 / 时间 / 对象 / 操作）
3. VIP 体系最小闭环：等级 / 有效期展示、模拟开通流程（前端状态切换 + 后端简易逻辑）
4. 权限基础管控：未登录不可发帖、普通用户不可执行治理操作
5. 核心功能联调、基础 bug 修复，形成可完整演示的 MVP 版本
6. 迭代增强（纳入 Sprint 3，低优先级，进度富余时开发，可择一实现）

Sprint 3（第 5-6 周）：功能扩展与体验优化

1. 互动功能增强：转发、收藏、关注、通知中心（未读 / 已读）
2. 内容检索功能：全站 / 分区内关键词搜索
3. 分区能力增强：分区模板、二级分区自治审核（父分区管理员初审）
4. 亮点模块深入实现（择一）：问答模块（提问 / 回答 / 采纳）或二手交易模块（商品发布 / 浏览）
5. 界面体验优化、高级 bug 修复、文档完整迭代
7. 冲刺交付标准

每个 Sprint 结束后，必须交付**可独立运行、可演示的功能版本+对应模块的文档更新**，通过团队内部评审后，进入下一冲刺阶段；若未达到交付标准，顺延 1-2 天进行修复，确保核心进度不滞后。

本系统的功能规划采用“必做核心闭环 + 迭代增强扩展”的双层策略，确保在课程项目周期内优先实现可运行的完整演示版本，同时为后续功能深化预留明确方向。在必做核心功能层面，系统将围绕用户体系与内容生态构建最小可行产品闭环：用户注册登录与个人主页基础信息（头像、背景、主题、ID）构成用户身份的基石；分区与子分区浏览机制支持用户按主题层级探索内容；图文发帖功能涵盖内容发布、列表展示与详情页呈现，形成完整的内容消费链路；点赞与评论（支持至少二级评论）实现基础互动能力；建区申请与审核机制作为平台可生长特性的核心体现，支持用户提交分区创建申请，审核通过后系统自动创建分区并初始化预设功能模块；后台治理功能涵盖内容下架、置顶操作及审计日志记录，确保平台具备基础的管理与追溯能力；VIP 体系以等级与到期时间展示为前端呈现，配合模拟开通流程形成最小闭环，体现用户差异化体验。在上述核心功能稳定运行的基础上，系统规划了迭代增强方向以拓展平台深度与社区活力：转发、收藏、关注与通知中心将进一步丰富用户互动场景与消息感知；全站及分区内搜索功能提升内容发现效率；分区模板机制支持快速复用已有分区配置，二级分区自治审核则探索分区层面的权限下放与自主治理；此外，项目团队需从问答社区、二手交易、活动组织三大方向中选择其一作为亮点模块进行深入实现，在核心架构之上验证系统的模块扩展能力，展示可配置驱动的设计理念。通过上述分层规划，本系统既确保基础业务流程完整可运行，又为功能迭代与技术亮点呈现提供了清晰路径，全面契合课程设计对工程规范与创新实践的雙重要求。

第7章 附录 B：引用文件

- 课程实验指导：《实验内容一(1)》
- 课程实验指导：《实验内容二》
- 课程实验指导：《实验内容三》
- 课程可选课题：“博客网站”方向描述
- 软件工程常用文档规范（课程允许参考）
- 团队协作规范（GitHub Flow、接口文档规范、代码规范等）
- 《Scrum 理论与实践的轻量级指南》

第8章附录 C：Scrum 开发过程模型落地设计

8.1 Scrum 核心角色分配

- 产品负责人（Product Owner）：1 人，负责梳理 Product Backlog、确定功能优先级、对接课程老师 / 助教获取需求反馈，把控项目整体方向；
- Scrum Master：1 人（可由产品负责人兼任），负责组织 Scrum 活动（每日站会、冲刺规划、冲刺评审）、解决开发过程中的障碍、跟踪风险与进度，确保 Scrum 流程正常执行；
- 开发团队（Development Team）：全体成员，按技能分为前端开发、后端开发、文档开发小组，负责执行 Sprint Backlog 中的任务、完成功能开发与测试、同步开发进度。

8.2 Scrum 核心活动执行规范（适配课程项目）

- 冲刺规划会议（每个冲刺开始前，30-60 分钟）：全团队参与，确定本次冲刺目标、选取 Sprint Backlog 任务、明确任务负责人与时间节点，形成冲刺计划；
- 每日站会（开发期间每日上午，10-15 分钟）：全体成员同步 3 个问题 ——“昨天完成了什么”“今天要做什么”“遇到了什么障碍”，Scrum Master 记录障碍并协调解决；
- 冲刺评审会议（每个冲刺结束后，30-60 分钟）：全团队参与，演示本次冲刺的交付成果，内部评审功能是否达标，收集问题并记录至下一个冲刺的待办列表；
- 冲刺复盘会议（冲刺评审后，15-30 分钟）：全团队参与，复盘本次冲刺的优点与问题，优化下一个冲刺的流程与执行方式。

8.3 Scrum 核心工件管理

- 产品待办列表（Product Backlog）：采用 GitHub Projects 或在线表格（如飞书表格、腾讯文档）管理，按 “优先级（高 / 中 / 低）+ 任务描述 + 负责人 + 预估工时” 梳理，动态更新；
- 冲刺待办列表（Sprint Backlog）：从 Product Backlog 中选取任务，明确本次冲刺的可交付成果，采用看板形式管理，实时更新任务状态（待开始 / 进行中 / 已完成 / 阻塞）；
- 增量（Increment）：每个冲刺结束后的可运行产品版本，是本次冲刺所有完成任务的总和，必须满足 “可演示、可运行” 的基本要求，作为课程阶段性验收的依据。